TÀI LIỆU HƯỚNG DẪN SỬ DỤNG

Coral

Sprite atlas & 2D animation toolkit

Coral là bộ công cụ desktop cho sprite 2D và animation: **đóng gói sprite thành atlas** rồi xuất cho Godot, Unity, Spine, Phaser / PixiJS (Pack), **cắt sprite sheet** có sẵn (Slice), **xem animation, đặt pivot, vẽ hitbox** (Animate), **ghép chuỗi frame VFX thành sheet SubUV** (Flipbook), **công cụ pixel art** — phóng to, palette, đổi màu (Pixel) — và **tạo normal map** cho ánh sáng 2D (Normal 2D).

Phiên bản: 0.1.1 · **Ngày tạo:** 2026-11-09

Nội dung này giống hệt phần Hướng dẫn trong ứng dụng (phím F1).

Mục lục

1 **Bắt đầu**

Coral là gì, bố cục giao diện, kéo & thả, project, cách Coral ghi file an toàn, phím tắt, log và giao diện.

2 **Pack — đóng gói atlas**

Gom PNG / TGA, Aseprite, PSD, GIF thành atlas (trim, gộp trùng, MaxRects) rồi xuất JSON (Phaser, PixiJS, Paper2D), Godot, Unity, Spine. Có Watch để tự xuất lại khi lưu.

3 **Slice — cắt sprite sheet**

Cắt một sheet có sẵn theo lưới hoặc tự động, sửa rect, tên, pivot, 9-slice; ghi PNG riêng lẻ hoặc metadata (JSON Hash, Unity .meta) cho sheet gốc.

4 **Animate — animation, pivot, hitbox**

Xem animation của project đang mở trong Pack, chỉnh FPS / kiểu lặp / thời lượng từng frame, onion skin, đặt pivot và vẽ hitbox / hurtbox theo từng frame.

5 **Flipbook — sheet SubUV cho VFX**

Ghép chuỗi frame đánh số thành một sheet N×M đều nhau (SubUV) cho Niagara / Unity particle, kèm JSON tuỳ chọn.

6 **Pixel — phóng to, palette, đổi màu, bleed**

Bốn công cụ hàng loạt cho pixel art: Upscale (nearest neighbour), Remap về palette (OKLab), Swap tạo biến thể màu, Bleed tô màu pixel trong suốt.

7 **Normal 2D — normal map cho ánh sáng 2D**

Tạo normal map từ độ sáng hoặc vát viền alpha, chuẩn OpenGL / DirectX, xem trước có ánh sáng theo con trỏ; dùng cùng thiết lập cho normal atlas của Pack.

1 Bắt đầu

Coral là gì, bố cục giao diện, kéo & thả, project, cách Coral ghi file an toàn, phím tắt, log và giao diện.

Coral là gì?

Coral là bộ công cụ desktop cho sprite 2D và animation: **đóng gói sprite thành atlas** rồi xuất cho Godot, Unity, Spine, Phaser / PixiJS (Pack), **cắt sprite sheet** có sẵn (Slice), **xem animation, đặt pivot, vẽ hitbox** (Animate), **ghép chuỗi frame VFX thành sheet SubUV** (Flipbook), **công cụ pixel art** — phóng to, palette, đổi màu (Pixel) — và **tạo normal map** cho ánh sáng 2D (Normal 2D).

- Giao diện ứng dụng bằng tiếng Anh; hướng dẫn này in **đậm** đúng tên nút / tuỳ chọn như trên màn hình.
- Mở hướng dẫn bất cứ lúc nào bằng `F1` (mở đúng chủ đề của module đang dùng), nút **?** trên header, hoặc Ctrl+K → **Help / Hướng dẫn**.

Bố cục giao diện

- **Thanh điều hướng (cột trái)**: 6 module theo thứ tự Ctrl+1 … Ctrl+6 — Pack, Slice, Animate, Flipbook, Pixel, Normal 2D. Phía dưới: **Settings** (Ctrl+,), **About & contact**, **Command palette** (Ctrl+K) và nút **Show log / Hide log** (có huy hiệu đếm lỗi khi log đang đóng).
- **Header**: `Coral / <module>`, tên project (`(unsaved)` khi chưa lưu, dấu `•` khi có thay đổi chưa lưu), ô **Search actions…** (mở Command Palette), nút **?** (hướng dẫn này) và nhãn phiên bản (`browser preview` khi chạy ngoài app desktop; `via Abyss` khi được Abyss khởi chạy).
- **Log (dưới đáy)**: nhật ký của giao diện và phần xử lý Rust. Bật / tắt bằng Ctrl+`; Ctrl+K → **Clear log** để xoá.

Mở file và thư mục

- **Kéo & thả** file hoặc thư mục vào bất kỳ đâu trên cửa sổ: chúng được nạp vào **module đang mở**. Mỗi module nhận một loại đầu vào khác nhau (xem chủ đề của từng module).
- Pack và Animate: thả thư mục / `.aseprite` / `.psd` / `.gif` để thêm nguồn; thả một file `*.coral.json` để mở project đó.
- Slice: một sheet PNG / TGA / GIF. Flipbook: một thư mục frame. Pixel và Normal 2D: ảnh hoặc thư mục (được thêm vào danh sách đầu vào).
- Ctrl+K → **Add a folder…** chọn một thư mục và gửi nó tới module đang mở, như khi thả vào cửa sổ.

Project (*.coral.json)

Project lưu những gì Pack và Animate dùng: nguồn sprite, thiết lập pack, các đích xuất (exports), pivot đã sửa, animation đã sửa, hitbox và bảng swap màu (nếu bạn bấm **Save to project** trong Pixel). Đường dẫn trong file là **tương đối** so với file project, nên có thể để project trong repo game.

- **Open…** / **Save** ở thanh trên cùng của Pack, hoặc Ctrl+O / Ctrl+S / Ctrl+Shift+S (lưu thành file khác). Lần lưu đầu tiên hỏi nơi lưu, tên mặc định `<atlas>.coral.json`.

- Ctrl+K → **New project** bắt đầu project trống (dừng Watch nếu đang chạy; không hỏi lưu — hãy lưu trước). Mục **Recent projects** trong Command Palette mở lại 10 project gần nhất.
- Thiết lập của Slice, Pixel và Normal 2D **không** nằm trong project. Tuỳ chọn của Flipbook (trừ danh sách frame) được ứng dụng nhớ giữa các lần mở.

Coral ghi file thế nào

- **Pack** và **Slice** luôn hiện danh sách file sẽ ghi trước (hộp thoại **Write N file(s)?**, mỗi dòng gắn nhãn `will write`); chỉ khi bấm **Write files** mới ghi.
- **Flipbook**, **Pixel** và **Normal 2D** ghi ngay khi bấm nút chính. Pixel và Normal 2D có nút **Dry run** để xem danh sách trước; Flipbook tự lập kế hoạch lại mỗi khi bạn đổi tuỳ chọn.
- Mọi file được ghi **nguyên tử**: ghi ra `<file>.tmp` rồi đổi tên. File đã có cùng đường dẫn sẽ bị **thay thế**.
- Chế độ **In place (.bak)** (Pixel) sao chép từng file gốc sang `<file>.bak` trước khi ghi đè. Nếu `.bak` đã tồn tại thì giữ nguyên `.bak` cũ (bản gốc đầu tiên được bảo toàn).
- Coral không bao giờ xoá file của bạn. Bấm vào đường dẫn của file đã ghi trong danh sách kết quả để mở nó trong Explorer.

Phím tắt

`F1`	Mở / đóng hướng dẫn này (đúng chủ đề của module đang mở)
`Ctrl` `K`	Command Palette: chuyển module, thao tác project, project gần đây, cập nhật…
`Ctrl` `1` … `Ctrl` `6`	Chuyển module (Pack, Slice, Animate, Flipbook, Pixel, Normal 2D)
`Ctrl` `O`	Mở project
`Ctrl` `S` / `Ctrl` `Shift` `S`	Lưu project / lưu thành file khác
`F5`	Chạy lại module hiện tại (không bao giờ ghi file): Pack / Animate pack lại bản xem trước, Slice **Detect** lại, Flipbook lập kế hoạch lại, Normal 2D làm mới bản xem trước; Pixel không làm gì
`Ctrl` `` ` ``	Hiện / ẩn log
`Ctrl` `,`	Settings (giao diện)
`Esc`	Đóng hộp thoại / Command Palette / hướng dẫn

- Trong Slice, `F5` chạy lại **Detect** và **thay toàn bộ** các rect — mọi chỉnh sửa tay (vị trí, tên, pivot, 9-slice) sẽ mất.

Giao diện và cập nhật

- **Settings** (Ctrl+,): chủ đề sáng / tối / theo Windows, font giao diện và font code, cỡ giao diện, mức hiệu ứng.

- Ctrl+K → **Check for updates…** (hoặc trong **About & contact**). Khi có bản mới, header hiện nút **Update to vX.Y.Z** để tải, cài và khởi động lại.
- Khi được Abyss khởi chạy, Coral không tự kiểm tra cập nhật: Abyss báo và cài bản mới khi ứng dụng đóng.

2 Pack — đóng gói atlas

Gom PNG / TGA, Aseprite, PSD, GIF thành atlas (trim, gộp trùng, MaxRects) rồi xuất JSON (Phaser, PixiJS, Paper2D), Godot, Unity, Spine. Có Watch để tự xuất lại khi lưu.

Cách dùng

1. **Thêm nguồn**: thả thư mục PNG / TGA, file `.aseprite` / `.ase`, `.psd` hoặc `.gif` vào cửa sổ, hoặc bấm nút thư mục (**Add a folder**) trên thẻ **Sources**. Thả một PNG lẻ sẽ thêm **thư mục chứa nó** (không gồm thư mục con, chỉ loại file đó).
2. Đặt **tên atlas** ở ô bên trái thanh trên cùng (dùng cho tên file xuất). Project chưa lưu tự lấy tên của nguồn đầu tiên.
3. Xem **atlas** ở giữa: nó được pack lại sau mỗi chỉnh sửa. Chỉnh **Pack** settings bên phải.
4. Trong thẻ **Exports**, bấm +, chọn đích (**JSON Hash**, **JSON Array**, **Godot**, **Unity**, **Spine**) rồi chọn thư mục xuất.
5. Bấm **Pack & Export**: Coral liệt kê các file sẽ ghi; bấm **Write files** để ghi.
6. **Save** (Ctrl+S) để lưu project, rồi bật **Watch** nếu muốn atlas tự xuất lại mỗi khi bạn lưu file nguồn.

Nguồn (Sources)

Bấm vào một nguồn để sửa nó; nút × bỏ nguồn khỏi project (không xoá file).

- **Thư mục**: **Include subfolders** (bật sẵn khi thả thư mục) và danh sách mẫu tên, cách nhau bằng dấu phẩy (mặc định `*.png, *.tga`).
- **Aseprite**: **Visible merged** (gộp các layer đang hiện), **All merged** (gộp mọi layer) hoặc **Per layer** — gõ tên các layer cần tách, cách nhau bằng dấu phẩy (ví dụ `Body, FX`); mỗi layer thành một bộ sprite riêng. Tag thành animation; thời lượng frame, pivot và 9-slice từ slice được giữ.
- **PSD**: **One sprite per layer** (mặc định) hoặc **Flattened** (cả file thành một sprite). Chỉ đọc layer pixel (không đọc effect / smart object).
- **GIF**: mỗi frame là một sprite `<file>_<i>`.
- Tên sprite: đường dẫn trong thư mục, bỏ đuôi, thư mục con nối bằng `/` (ví dụ `fx/spark_0`); Aseprite: `<file>_<tag>_<i>`. Hai sprite trùng tên là lỗi `duplicate-name` — Coral không tự đổi tên.
- Animation tự nhận từ tag Aseprite và tên đánh số (`run_0, run_1…`, `walk0001…`). Xem và sửa chúng trong module **Animate**.

Xem atlas

- Zoom 25 %–800 % bằng các nút phía trên. Rê chuột lên một sprite để thấy tên; bấm để chọn — thanh dưới hiện frame trong atlas, kích thước gốc, vùng đã trim và pivot.
- Khi một trang không đủ chỗ, các trang hiện dưới dạng **Page 1**, **Page 2**… Nếu bật **Normal-map atlas**, ô **Normal map** cho xem atlas normal.
- Danh sách **Sprites** bên trái có ô **Filter**…. Nhãn `alias` = sprite có pixel giống hệt sprite khác: chúng dùng chung một ô trong atlas.

- Thanh dưới cùng: số sprite, trang, alias, % lấp đầy, số animation và thời gian pack bản xem trước.

Thiết lập Pack

- **Max size** (512–8192, mặc định 2048): cạnh tối đa của một trang. **Image**: **PNG** hoặc **TGA**.
- **Padding** (mặc định 2 px): khoảng trống giữa các sprite. **Extrude** (mặc định 1 px): lặp lại pixel viền ra ngoài để lọc texture không kéo màu của sprite bên cạnh.
- **Trim** cắt viền trong suốt; **Trim alpha** ≤ coi pixel có alpha ≤ giá trị này là trống. Kích thước gốc và offset vẫn được xuất, nên sprite không bị lệch trong engine.
- **Merge duplicates**: sprite giống hệt nhau dùng chung một ô (alias). **Power of two** / **Square**: kích thước trang là luỹ thừa của 2 / hình vuông. **Multiple pages**: thêm trang khi một trang không đủ (tắt thì sprite không vừa là lỗi).
- **Allow rotation**: xoay sprite 90° để xếp chặt hơn — **Godot và Unity không hỗ trợ**: xuất cho chúng sẽ bị chặn.
- **Alpha bleed**: tô pixel trong suốt bằng màu lân cận (alpha vẫn 0) để không có viền tối khi lọc / mipmap. **Premultiply**: nhân trước màu với alpha (chỉ bật nếu engine của bạn cần).
- **Normal-map atlas**: thêm atlas thứ hai `<name>_n` với cùng các ô; thiết lập lấy từ module **Normal 2D** (nút **Use for the project's normal atlas**).
- **Placement**: cách MaxRects chọn chỗ — **Best short side** (mặc định), **Best long side**, **Best area**, **Bottom left**, **Contact point**. Thử vài cách nếu muốn atlas nhỏ hơn.

Đích xuất (Exports) và file tạo ra

Mỗi đích ghi **ảnh atlas riêng của nó** vào thư mục của nó (`<name>.png`, hoặc `<name>-0.png`, `<name>-1.png` … khi có nhiều trang; `<name>_n.png` cho normal atlas) cùng file dữ liệu bên dưới. `<name>` là tên atlas.

- **JSON Hash** (định dạng TexturePacker — Phaser 3, PixiJS): `<name>.json`. Tick **Unreal Paper2D (.paper2dsprites)** để file dữ liệu có đuôi `.paper2dsprites` cho Paper2D. **JSON Array**: cùng nội dung, frames là mảng. FPS, kiểu lặp, thời lượng frame, hitbox và 9-slice nằm trong `meta.coral`.
- **Godot** (Godot 4): `<name>.tres` (SpriteFrames chứa mọi animation; ping-pong được trải ra thành frame). Thư mục xuất phải nằm trong một project Godot — `project.godot` được tìm ngược lên các thư mục cha, hoặc bấm dòng **Godot project:** để chỉ định. Tuỳ chọn **Also one AtlasTexture .tres per sprite** cho sprite UI.
- **Unity** (Unity 6): `<name>.png.meta` (Sprite Mode Multiple) với **Pixels per unit** (mặc định 100) và **Filter Point** / **Bilinear**. Xuất lại **giữ nguyên guid** của `.meta`, nên tham chiếu trong scene không bị mất.
- **Spine** (Spine / libGDX): `<name>.atlas`, filter **Linear** hoặc **Nearest**.
- Bấm vào đường dẫn của một đích để đổi thư mục; biểu tượng thùng rác bỏ đích đó. Đích chưa có thư mục hiện **Choose a folder…** màu đỏ và sẽ chặn việc xuất.

Pack & Export

1. Bấm **Pack & Export** (cần ít nhất một đích xuất).
2. Hộp thoại **Write N file(s)?** liệt kê mọi file (`will write`). File cùng đường dẫn sẽ bị thay thế; `.meta` Unity có sẵn giữ guid.
3. Bấm **Write files**. Thẻ **Last export** hiện kết quả và thời gian; bấm một đường dẫn để mở trong Explorer.

- **Export blocked**: có lỗi (nguồn không đọc được, trùng tên sprite, sprite lớn hơn atlas, xoay sprite khi xuất Godot / Unity, thiếu thư mục…). Không file nào được ghi; thẻ **Issues** nói rõ lỗi gì và ở đâu.

Watch

- **Watch** bật được khi project có nguồn và ít nhất một đích xuất. Nó pack và xuất lại mỗi khi một file nguồn (hoặc file project đã lưu) thay đổi — lưu trong Aseprite là engine tự nạp lại.
- Watch **ghi file mà không hỏi lại** mỗi lần chạy, vào đúng các thư mục xuất đã đặt.
- Nhãn trạng thái hiện lần chạy gần nhất; chuyển đỏ khi lỗi — bấm vào để mở log. **Stop** để dừng. Watch dừng khi đóng ứng dụng; `"watch": { "autoStart": true }` trong project sẽ tự bật khi mở project.

Dòng lệnh

Cùng file exe chạy không giao diện khi tham số đầu là một lệnh, ví dụ `coral.exe pack hero.coral.json` (mọi đích xuất của project), thêm `--target godot --dry-run` để chỉ liệt kê, `--watch` để theo dõi, `--report pack.json` để ghi báo cáo. Xem `coral.exe --help` và README của Coral.

3 Slice — cắt sprite sheet

Cắt một sheet có sẵn theo lưới hoặc tự động, sửa rect, tên, pivot, 9-slice; ghi PNG riêng lẻ hoặc metadata (JSON Hash, Unity .meta) cho sheet gốc.

Cách dùng

1. Thả một sheet PNG / TGA / GIF vào cửa sổ, hoặc bấm **Choose a sheet...**. Coral chạy **Detect** ngay (chế độ **Automatic**).
2. Chọn **Mode**: **Automatic** hoặc **Grid**, chỉnh thông số rồi bấm **Detect**.
3. Sửa rect trên sheet (xem bên dưới), đặt tên, pivot và 9-slice trong thẻ **Rect N**.
4. Trong thẻ **Write**, chọn kiểu đầu ra rồi bấm **Slice N sprite(s)...**: Coral liệt kê file sẽ ghi; bấm **Write files** để ghi.

Automatic và Grid

- **Automatic**: tìm từng hình tách rời (các pixel có alpha > **Alpha** >, nối 8 hướng). Hình cách nhau ≤ **Merge within** px (mặc định 2) gộp thành một sprite; hình nhỏ hơn **Ignore under** pixel (mặc định 4) bị bỏ qua. Rect được xếp theo thứ tự đọc: hàng trên xuống dưới, trái sang phải.
- **Grid**: cắt ô cố định **Cell W** × **Cell H** (mặc định 32 × 32), bắt đầu từ **Offset X / Y**, với khoảng cách **Gap X / Y** giữa các ô. Ô trống bị bỏ trừ khi bật **Keep empty cells**.
- **Detect** (và `F5`) thay **toàn bộ** rect hiện có — chỉnh sửa tay sẽ mất. Hãy chọn chế độ và thông số trước, rồi mới sửa tay.

Sửa rect

- Kéo một rect để di chuyển; kéo góc dưới-phải của rect đang chọn để đổi kích thước. Số thứ tự hiện trên rect khi zoom từ 200 %.
- **+ Rect** rồi kéo trên sheet để thêm rect mới. Biểu tượng thùng rác trên thẻ **Rect N** xoá rect đang chọn.
- Thẻ **Rect N**: tên, **X / Y / W / H** chính xác, **Pivot** (9 vị trí sẵn hoặc giá trị X / Y từ 0 đến 1; **Use this pivot for every rect** áp cho mọi rect) và **9-slice borders** (**L / T / R / B**, hiện bằng nét đứt xanh).

Đặt tên hàng loạt

Thẻ **Names**: mẫu tên như `{base}_{i}` — `{base}` = tên sheet, `{i}` = số thứ tự từ 0, `{i:03}` = số có đệm 0 (`007`), `{i1}` = đánh số từ 1. Bấm **Rename all** để áp dụng cho mọi rect (Detect cũng dùng mẫu này).

Đầu ra

- **Separate PNGs**: mỗi rect thành một PNG trong **Output folder** (mặc định `<sheet>_sprites` cạnh sheet).
- **JSON Hash**: `<sheet>.json` mô tả các rect của sheet gốc (không cắt ảnh), ghi vào **Output folder**.
- **Unity .meta**: `<sheet>.png.meta` **cạnh sheet** — Unity tự cắt khi import.
- **Cannot slice**: hai rect trùng tên hoặc rect nằm ngoài sheet. Sửa rồi thử lại.

- Rect, tên và pivot của Slice không được lưu vào project và mất khi mở sheet khác — hãy ghi xong trước khi chuyển sheet.

4 Animate — animation, pivot, hitbox

Xem animation của project đang mở trong Pack, chỉnh FPS / kiểu lặp / thời lượng từng frame, onion skin, đặt pivot và vẽ hitbox / hurtbox theo từng frame.

Animation đến từ đâu

Animate dùng project đang mở trong **Pack** (atlas xem trước của nó). Animation tự nhận từ tag Aseprite và tên đánh số (`run_0…`, `walk0001…`). Chưa có nguồn thì thêm ở Pack trước — thả nguồn vào Animate cũng thêm vào project.

> - Sửa một animation sẽ lưu nó vào project (nhãn **edited** trong danh sách). Nhớ **Save** project (Ctrl+S) để giữ thay đổi.

Trình phát

> - **Play** / **Pause**, ◀ ▶ để lùi / tiến một frame. Chọn animation ở danh sách **Animations** bên trái (số frame · FPS).
> - Thẻ **Playback**: **FPS**, **Loop** (**Loop**, **Once**, **Ping-pong**) và **Onion skin** (0–3 frame trước và sau, mờ dần từ 30 %).
> - Dải frame dưới khung hình: bấm để nhảy tới frame; ô số dưới mỗi frame là thời lượng (ms) — gõ số để đổi riêng frame đó. **Same duration for every frame** bỏ thời lượng riêng, quay về FPS.
> - Chấm đỏ dưới một frame nghĩa là frame đó có box.

Pivot

1. Chọn công cụ hình tâm ngắm (**Click to set the pivot**) rồi bấm hoặc kéo trên khung hình — hoặc dùng 9 vị trí sẵn trong thẻ **Pivot**.
2. **Apply to the whole animation** (bật sẵn) đặt pivot cho mọi frame; tắt để chỉ đổi frame hiện tại.

> - Pivot được lưu dạng chuẩn hoá 0–1 theo sprite **chưa trim** và được xuất sang mọi đích (JSON, Godot, Unity, Spine).

Hitbox, hurtbox, box tuỳ chỉnh

1. Trong thẻ **Boxes · frame N**, chọn **New box type**: **Hitbox** (đỏ), **Hurtbox** (xanh lá) hoặc **Custom** (vàng, gõ tên).
2. Chọn công cụ hình vuông (**Drag to draw a box**) rồi kéo trên khung hình.
3. Dùng công cụ mũi tên (**Select a box**) để chọn box; đặt tên hoặc xoá nó trong danh sách.
4. Nút sao chép (**Copy boxes to the next frame**) chép mọi box của frame này sang frame kế tiếp và chuyển tới đó — tiện cho chuỗi đòn đánh.

> - Box được xuất trong `meta.coral.hitboxes` của các file JSON (JSON Hash / Array): toạ độ pixel của sprite **chưa trim**, gốc ở góc trên-trái. `docs/FORMATS.md` có ví dụ đọc chúng trong Godot và Unity.

5 Flipbook — sheet SubUV cho VFX

Ghép chuỗi frame đánh số thành một sheet N×M đều nhau (SubUV) cho Niagara / Unity particle, kèm JSON tuỳ chọn.

Cách dùng

1. Thả một thư mục frame đánh số (`explosion_0001.png … explosion_0064.png`) hoặc dùng **Add folder…** trong thẻ **Frames**. Frame được sắp theo thứ tự tự nhiên (2 trước 10).
2. Chọn frame: **From** / **To** (đánh số từ 1; 0 = đầu / cuối) và **Step** (lấy 1 frame mỗi N frame).
3. Thẻ **Grid**: **Columns** / **Rows** (0 = Coral gợi ý lưới gần vuông nhất), **Cell** (cạnh dài của ô, px; 0 = giữ kích thước frame; thu phóng bằng Lanczos3) và **Max size** (1024–8192).
4. Xem sheet và trình phát (chạy từng ô theo **FPS**). Bên dưới là thông số cho engine: Niagara SubUV **Sub Image Size** = cột × hàng; Unity Texture Sheet Animation **Tiles X / Y**.
5. Kiểm tra **Output image** rồi bấm **Write flipbook**.

Đầu ra

- **Output image** mặc định là `T_<Thư mục>_SubUV.png` cạnh thư mục frame (ví dụ `T_Explosion_SubUV.png`). Đổi đuôi thành `.tga` để ghi TGA.
- **Also write a .json (cols, rows, frames, fps, cell)**: ghi thêm `<tên>.json` cạnh ảnh.
- **Write flipbook** ghi ngay, không có hộp thoại xác nhận, và thay thế file cùng tên nếu đã có.

Lưu ý

- Không trim, không padding: mọi ô cùng kích thước. Frame khác kích thước được co về kích thước của frame đầu tiên (có cảnh báo).
- Nếu sheet vượt **Max size**, các ô tự thu nhỏ cho vừa. Lưới quá nhỏ so với số frame thì các frame cuối bị bỏ (có cảnh báo).
- Ô được điền trái sang phải, trên xuống dưới. Tuỳ chọn được nhớ giữa các lần mở ứng dụng (trừ danh sách frame).

6 Pixel — phóng to, palette, đổi màu, bleed

Bốn công cụ hàng loạt cho pixel art: Upscale (nearest neighbour), Remap về palette (OKLab), Swap tạo biến thể màu, Bleed tô màu pixel trong suốt.

Cách dùng chung

1. Chọn công cụ trong thẻ **Tool**: **Upscale**, **Remap**, **Swap** hoặc **Bleed**.
2. Thêm đầu vào: thả ảnh / thư mục (thư mục gồm cả thư mục con) hoặc **Add folder…** / **Add file…**. **Before / after** hiện ảnh đầu tiên và kết quả của nó.
3. Chọn **Output**: **Folder** (một thư mục; để trống = cạnh từng file gốc) hoặc **In place (.bak)** (ghi đè file gốc sau khi sao lưu sang `<file>.bak`).
4. Bấm **Dry run** để xem danh sách file, rồi bấm nút chính (ví dụ **Upscale ×4**) để ghi. Nút chính ghi ngay, không hỏi lại.

Upscale

Phóng to theo nearest neighbour, **Factor** ×2, ×3, ×4, ×5, ×6 hoặc ×8 — pixel giữ sắc nét. File ra: `<tên>_x4.png` (theo hệ số, giữ định dạng PNG / TGA của file gốc).

Remap (đưa về palette)

Mỗi màu được thay bằng màu gần nhất trong palette (đo trong không gian OKLab, alpha giữ nguyên). File ra: `<tên>_remap.png`. Kết quả liệt kê số pixel đã đổi và màu nào bị gộp vào màu nào. Nút **Remap to palette** cần palette có ít nhất một màu.

> - **Extract from inputs**: lấy palette từ chính các ảnh đầu vào, sắp **By use** (dùng nhiều nhất trước) hoặc **By hue**. Tối đa 4096 màu.
> - Nút tải lên (**Load palette**) mở Lospec `.hex`, GIMP `.gpl` hoặc JASC `.pal`; nút tải xuống (**Save palette**) lưu palette.
> - **+** thêm màu; bấm một ô màu để bỏ nó khỏi palette.

Swap (biến thể màu)

Mỗi bảng swap là một biến thể: danh sách cặp màu `từ → đến`. Mỗi bảng tạo file `<tên>_<biến thể>.png` và **không bao giờ đụng vào file gốc**: biến thể được ghi vào **Output folder**, hoặc cạnh file gốc khi để trống.

1. Lấy palette (**Extract from inputs** hoặc tải file) — các ô màu của palette hiện trong mỗi bảng.
2. Đặt tên biến thể (ô bên trái, ví dụ `alt`), bấm một ô màu để thêm cặp, rồi chọn màu đích. **+ pair** thêm cặp trống; **×** bỏ cặp.
3. **+** trên thẻ **Swap tables** thêm biến thể; thùng rác bỏ biến thể. Bấm **Make variants** để ghi.

> - **Save to project** lưu các bảng vào project đang mở (nhớ Ctrl+S); **From project** nạp lại chúng.

Bleed

Pixel trong suốt lấy màu của pixel lân cận, alpha vẫn bằng 0 — hết viền tối khi engine lọc texture hoặc tạo mipmap. Ảnh trông không đổi.

> - Khi ghi ra thư mục (kể cả để trống = cạnh file gốc), file ra có hậu tố `_bleed` (ví dụ `hero_bleed.png`), file gốc không bị đụng tới. Muốn sửa trực tiếp file gốc thì chọn **In place (.bak)**: Coral tạo bản `.bak` trước khi ghi.

7 Normal 2D — normal map cho ánh sáng 2D

Tạo normal map từ độ sáng hoặc vát viền alpha, chuẩn OpenGL / DirectX, xem trước có ánh sáng theo con trỏ; dùng cùng thiết lập cho normal atlas của Pack.

Cách dùng

1. Thả sprite (hoặc thư mục) vào cửa sổ, hoặc **Add folder…** / **Add file…**. Bản xem trước hiện sprite đầu tiên được chiếu sáng bởi một nguồn sáng theo con trỏ, cạnh normal map của nó; khi có nhiều ảnh, các nút tên file bên dưới dùng để chuyển ảnh.
2. Chọn **Height from**: **Luminance** (sáng = nổi) hoặc **Alpha bevel** (nổi dần từ mép sprite vào trong, rộng **Bevel** px — hợp với sprite tô màu phẳng).
3. Chỉnh **Strength** (độ dốc), **Blur** (làm mềm, px) và **Invert (dark = raised)** nếu cần.
4. Chọn **Green channel**: **OpenGL (Godot, Unity)** hoặc **DirectX (Unreal)**.
5. Trong thẻ **Write**, đặt **Output folder** (để trống = cạnh từng file gốc), bấm **Dry run** để xem danh sách, rồi **Write normal maps**.

Đầu ra

> - Mỗi ảnh đầu vào cho một `<tên>_n.png`. Pixel trong suốt là normal phẳng `(128, 128, 255)`, alpha giữ nguyên.
> - **Write normal maps** ghi ngay, không hỏi lại; file `_n.png` đã có sẽ bị thay thế. File gốc không bao giờ bị ghi đè (không có chế độ In place).

Normal atlas cho Pack

Use for the project's normal atlas chép thiết lập hiện tại vào project và bật **Normal-map atlas** trong Pack (nút hiện `(on)` khi đã bật): `hero_n.png` có đúng các ô như `hero.png`, nên một file mô tả atlas dùng được cho cả hai.

> - Rê chuột trên bản xem trước để di chuyển nguồn sáng và kiểm tra hướng nổi / lõm trước khi ghi.